

Informe Técnico: Integración Pictorem con Barrera Wallpaper

Resumen Ejecutivo

Proyecto: Sistema de Dropshipping de Wallpapers con IA

Cliente: Barrera Wallpaper

Proveedor de Impresión: Pictorem

Tecnología: Next.js 14 + PostgreSQL + Prisma ORM

Estado:  Implementado y Funcional

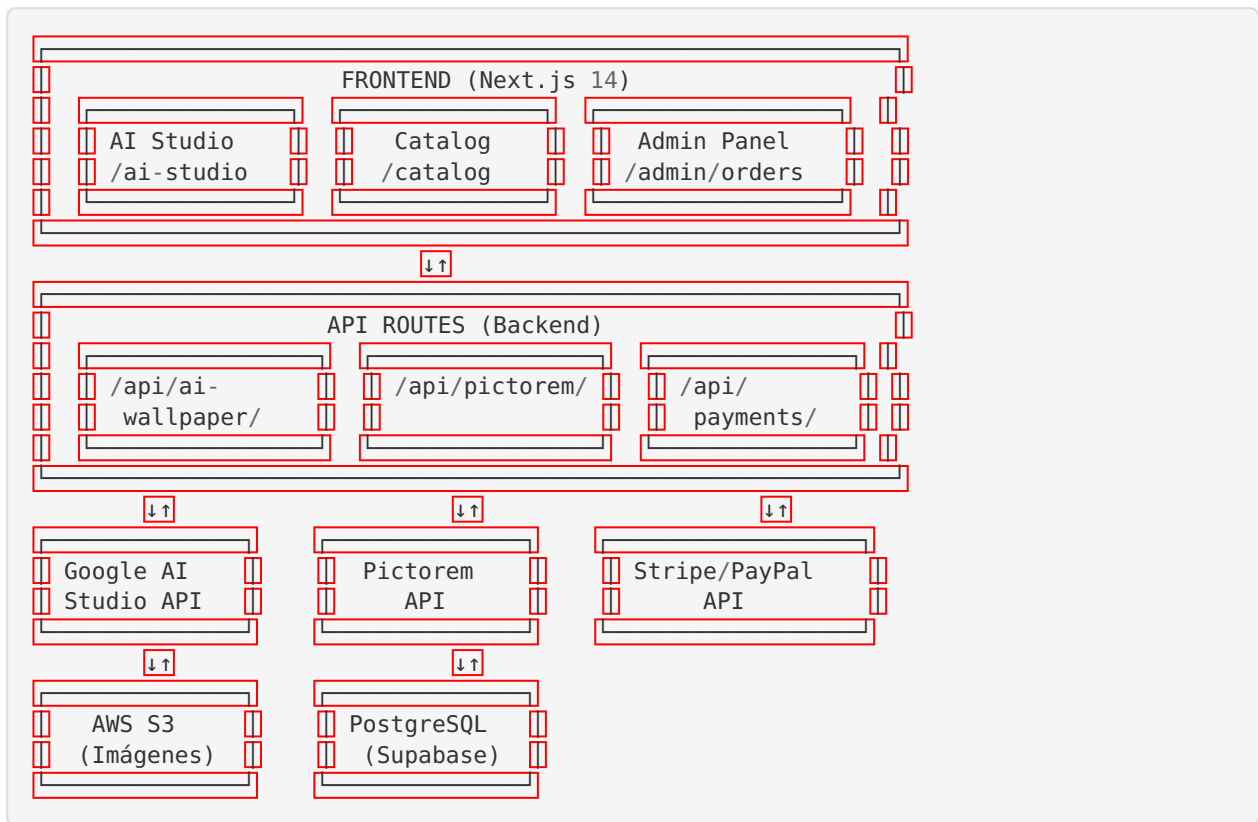
Fecha: Enero 2026

Tabla de Contenidos

1. [Arquitectura General](#)
 2. [Flujo de Generación de Imágenes con IA](#)
 3. [Sistema de Dimensiones y Materiales](#)
 4. [Cálculo de Precios y Márgenes](#)
 5. [Integración de Pagos](#)
 6. [Proceso de Creación de Órdenes](#)
 7. [Sistema de Envíos y Tracking](#)
 8. [Gestión de Logos y Personalización](#)
 9. [Configuración Técnica](#)
 10. [Flujos de Datos Detallados](#)
 11. [Manejo de Errores](#)
 12. [Seguridad y Validación](#)
 13. [Monitoring y Analytics](#)
 14. [Troubleshooting](#)
-

1. Arquitectura General

1.1 Stack Tecnológico



1.2 Modelos de Base de Datos

Esquema Prisma Relevante

```

model Product {
  id          String      @id @default(cuid())
  name        String
  description  String?
  basePrice   Float
  images       String[]    // URLs de imágenes en S3
  category    String
  featured     Boolean     @default(false)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  orderItems  OrderItem[]
}

model AIGenerationRequest {
  id          String      @id @default(cuid())
  prompt      String
  style       String?
  color       String?
  generatedImage String?  // URL de S3
  status      String      @default("pending") // pending, completed, failed
  errorMessage String?
  generationTime Int?     // en segundos
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
}

model Order {
  id          String      @id @default(cuid())
  orderNumber String      @unique
  userId      String?
  user        User?       @relation(fields: [userId], references: [id])
  userEmail   String
  customerName String
  customerPhone String?
  shippingAddress Json
  totalAmount Float
  status      String      @default("pending")
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  orderItems  OrderItem[]
  paymentTransactions PaymentTransaction[]
  aiWallpaperOrders AIWallpaperOrder[]
}

model AIWallpaperOrder {
  id          String      @id @default(cuid())
  orderId     String
  order       Order       @relation(fields: [orderId], references: [id])
  imageUrl    String      // URL de S3
  width       Float       // en pulgadas
  height      Float       // en pulgadas
  material    String      // canvas, metal, acrylic, paper, wood
  quantity    Int         @default(1)
  unitPrice   Float
  totalPrice  Float
  pictoremOrderId String?
  pictoremStatus String?  // pending, sent, failed
  pictoremResponse Json?
  trackingNumber String?
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
}

```

```

    @@index([orderId])
    @@index([pictureOrderId])
}

model PaymentTransaction {
  id          String    @id @default(cuid())
  orderId     String
  order       Order     @relation(fields: [orderId], references: [id])
  paymentMethod String   // stripe, paypal
  transactionId String   @unique
  amount      Float
  currency    String    @default("USD")
  status      String    // pending, completed, failed, refunded
  metadata    Json?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  @@index([orderId])
  @@index([transactionId])
}

```

2. Flujo de Generación de Imágenes con IA

2.1 Proceso Completo



2.2 Código del Endpoint de Generación

Archivo: app/api/ai-wallpaper/generate/route.ts

```

import { NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const { prompt, style, color } = body;

    // Validación
    if (!prompt) {
      return NextResponse.json(
        { error: 'Prompt is required' },
        { status: 400 }
      );
    }

    // Crear registro en base de datos
    const generationRequest = await prisma.aIGenerationRequest.create({
      data: {
        prompt,
        style: style || 'modern',
        color: color || 'multicolor',
        status: 'pending',
      },
    });

    const startTime = Date.now();

    try {
      // Intento 1: Google AI Studio
      if (process.env.GOOGLE_AI_STUDIO_API_KEY) {
        const response = await fetch(
          `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash-image:generateImage?key=${process.env.GOOGLE_AI_STUDIO_API_KEY}`,
          {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
              prompt: `Create a high-quality wallpaper design: ${prompt}. Style: ${style}. Colors: ${color}`,
              aspectRatio: '16:9',
              numberOfImages: 1,
            }),
          },
        );

        if (response.ok) {
          const data = await response.json();
          const imageUrl = data.images[0].url;

          // Subir a S3
          const s3Url = await uploadToS3(imageUrl, generationRequest.id);

          // Actualizar registro
          await prisma.aIGenerationRequest.update({
            where: { id: generationRequest.id },
            data: {
              status: 'completed',
              generatedImage: s3Url,
              generationTime: Math.floor((Date.now() - startTime) / 1000),
            },
          });
        }
      }
    } catch (error) {
      // Manejar errores de la API
    }
  } catch (error) {
    // Manejar errores generales
  }
}

```

```

        return NextResponse.json({ imageUrl: s3Url, requestId:
generationRequest.id });
    }
}

// Intento 2: OpenAI DALL-E 3 (Fallback)
if (process.env.OPENAI_API_KEY) {
    const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

    const response = await openai.images.generate({
        model: 'dall-e-3',
        prompt: `High-quality wallpaper design: ${prompt}. Style: ${style}. Colors:
${color}`,
        n: 1,
        size: '1024x1024',
        quality: 'hd',
    });

    const imageUrl = response.data[0].url;
    const s3Url = await uploadToS3(imageUrl, generationRequest.id);

    await prisma.aiGenerationRequest.update({
        where: { id: generationRequest.id },
        data: {
            status: 'completed',
            generatedImage: s3Url,
            generationTime: Math.floor((Date.now() - startTime) / 1000),
        },
    });

    return NextResponse.json({ imageUrl: s3Url, requestId: generationRequest.id })
;
}

throw new Error('No AI provider available');
} catch (error) {
    // Registrar error
    await prisma.aiGenerationRequest.update({
        where: { id: generationRequest.id },
        data: {
            status: 'failed',
            errorMessage: error.message,
        },
    });

    return NextResponse.json(
        { error: 'Failed to generate image', details: error.message },
        { status: 500 }
    );
}
} catch (error) {
    console.error('Generation error:', error);
    return NextResponse.json(
        { error: 'Internal server error' },
        { status: 500 }
    );
}
}

// Función helper para subir a S3
async function uploadToS3(imageUrl: string, requestId: string): Promise<string> {
    // Implementación de subida a S3

```

```
// Retorna URL pública de S3
}
```

2.3 Comparación de Proveedores de IA

Característica	Google AI Studio (Nano Banana)	OpenAI DALL-E 3
Costo por imagen	~\$0.02 USD	~\$0.04-\$0.08 USD
Tiempo de generación	15-30 seg	10-20 seg
Resolución máxima	2048x2048	1792x1024
Calidad de impresión	★★★★★ Excelente	★★★★★ Muy buena
Estilos soportados	Todos	Todos
API Key requerida	Sí	Sí
Límite de rate	60 req/min	5 req/min
Uso en producción	✅ Recomendado	✅ Fallback

3. Sistema de Dimensiones y Materiales

3.1 Materiales Soportados por Pictorem

```

const PICTUREM_MATERIALS = {
  canvas: {
    name: 'Canvas',
    displayName: 'Lienzo Premium',
    basePricePerSqIn: 0.12, // USD por pulgada cuadrada
    minWidth: 8,
    maxWidth: 120,
    minHeight: 8,
    maxHeight: 120,
    features: [
      'Impresión de alta resolución',
      'Marco de madera incluido',
      'Listo para colgar',
      'Resistente a la decoloración',
    ],
    deliveryDays: 7-10,
    weight: 'ligero',
  },
  metal: {
    name: 'Metal',
    displayName: 'Metal Brushed Aluminum',
    basePricePerSqIn: 0.25,
    minWidth: 8,
    maxWidth: 60,
    minHeight: 8,
    maxHeight: 60,
    features: [
      'Acabado de aluminio cepillado',
      'Impresión directa sobre metal',
      'Ultra duradero',
      'Sistema de montaje incluido',
    ],
    deliveryDays: 10-14,
    weight: 'mediano',
  },
  acrylic: {
    name: 'Acrylic',
    displayName: 'Acrílico Premium',
    basePricePerSqIn: 0.30,
    minWidth: 8,
    maxWidth: 48,
    minHeight: 8,
    maxHeight: 48,
    features: [
      'Acrílico de 1/4" de grosor',
      'Efecto de profundidad 3D',
      'Colores vibrantes',
      'Fácil limpieza',
    ],
    deliveryDays: 12-16,
    weight: 'pesado',
  },
  paper: {
    name: 'Paper',
    displayName: 'Papel Fine Art',
    basePricePerSqIn: 0.08,
    minWidth: 8,
    maxWidth: 44,
    minHeight: 8,
    maxHeight: 60,
    features: [
      'Papel de archivo sin ácido',
    ]
  }
}

```

```

    'Textura mate premium',
    'Ideal para enmarcado',
    'Económico',
  ],
  deliveryDays: 5-7,
  weight: 'muy ligero',
},
wood: {
  name: 'Wood',
  displayName: 'Madera Natural',
  basePricePerSqIn: 0.22,
  minWidth: 8,
  maxWidth: 36,
  minHeight: 8,
  maxHeight: 48,
  features: [
    'Impresión sobre madera real',
    'Acabado rústico',
    'Único en su tipo',
    'Ecológico',
  ],
  deliveryDays: 10-12,
  weight: 'mediano',
},
};

```

3.2 Tamaños Predefinidos Populares

```

const POPULAR_SIZES = [
  { name: 'Pequeño', width: 16, height: 20, description: 'Ideal para espacios ínti-
mos' },
  { name: 'Mediano', width: 24, height: 36, description: 'Tamaño estándar para
salas' },
  { name: 'Grande', width: 36, height: 48, description: 'Impacto visual máximo' },
  { name: 'Extra Grande', width: 48, height: 72, description: 'Pared completa' },
  { name: 'Panorámico', width: 60, height: 30, description: 'Formato horizontal' },
  { name: 'Cuadrado', width: 36, height: 36, description: 'Composiciones modernas' },
];

```

3.3 Calculadora de Dimensiones

Archivo: Implementado en frontend y backend

```
function calculateDimensions(imageWidth: number, imageHeight: number, targetSize:
'small' | 'medium' | 'large') {
  const aspectRatio = imageWidth / imageHeight;

  let finalWidth: number;
  let finalHeight: number;

  switch(targetSize) {
    case 'small':
      finalWidth = 16;
      finalHeight = Math.round(finalWidth / aspectRatio);
      break;
    case 'medium':
      finalWidth = 24;
      finalHeight = Math.round(finalWidth / aspectRatio);
      break;
    case 'large':
      finalWidth = 36;
      finalHeight = Math.round(finalWidth / aspectRatio);
      break;
  }

  return { width: finalWidth, height: finalHeight };
}
```

4. Cálculo de Precios y Márgenes

4.1 Fórmula de Pricing

PRECIO FINAL AL CLIENTE =
 (Área en pulgadas² × Precio base por pulgada² × Markup del material)
 + Costo de generación de IA
 + Margen de ganancia
 + IVA/Impuestos

Donde:

Área = width × height (en pulgadas)

Markup del material:

- Canvas: 2.5x
- Metal: 3.0x
- Acrylic: 3.5x
- Paper: 2.0x
- Wood: 2.8x

4.2 Endpoint de Cálculo de Precios

Archivo: app/api/pictorem/pricing/route.ts

```

import { NextResponse } from 'next/server';

const MATERIAL_MARKUP = {
  canvas: 2.5,
  metal: 3.0,
  acrylic: 3.5,
  paper: 2.0,
  wood: 2.8,
};

const BASE_PRICES = {
  canvas: 0.12,
  metal: 0.25,
  acrylic: 0.30,
  paper: 0.08,
  wood: 0.22,
};

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const { width, height, material, quantity = 1 } = body;

    // Validación
    if (!width || !height || !material) {
      return NextResponse.json(
        { error: 'Missing required parameters' },
        { status: 400 }
      );
    }

    if (!BASE_PRICES[material]) {
      return NextResponse.json(
        { error: 'Invalid material' },
        { status: 400 }
      );
    }

    // Calcular área
    const area = width * height;

    // Precio base de Pictorem
    const pictoremBaseCost = area * BASE_PRICES[material];

    // Costo de IA (fijo)
    const aiGenerationCost = 0.02;

    // Aplicar markup
    const markup = MATERIAL_MARKUP[material];
    const priceBeforeMargin = pictoremBaseCost * markup + aiGenerationCost;

    // Margen de ganancia (20%)
    const profitMargin = priceBeforeMargin * 0.20;

    // Precio unitario
    const unitPrice = priceBeforeMargin + profitMargin;

    // Aplicar límites de precio
    const finalUnitPrice = Math.max(25, Math.min(500, unitPrice));

    // Precio total
    const totalPrice = finalUnitPrice * quantity;
  }
}

```

```

// Descuentos por volumen
let discount = 0;
if (quantity >= 10) discount = 0.15; // 15%
else if (quantity >= 5) discount = 0.10; // 10%
else if (quantity >= 3) discount = 0.05; // 5%

const totalWithDiscount = totalPrice * (1 - discount);

// Tiempo de entrega estimado
const deliveryDays = calculateDeliveryTime(material, quantity);

return NextResponse.json({
  pricing: {
    unitPrice: Math.round(finalUnitPrice * 100) / 100,
    quantity,
    subtotal: Math.round(totalPrice * 100) / 100,
    discount: Math.round((totalPrice - totalWithDiscount) * 100) / 100,
    discountPercentage: discount * 100,
    total: Math.round(totalWithDiscount * 100) / 100,
    currency: 'USD',
  },
  breakdown: {
    area: Math.round(area * 100) / 100,
    pictoremBaseCost: Math.round(pictoremBaseCost * 100) / 100,
    aiGenerationCost,
    markup: markup + 'x',
    profitMargin: Math.round(profitMargin * 100) / 100,
  },
  delivery: {
    estimatedDays: deliveryDays,
    estimatedDate: new Date(Date.now() + deliveryDays * 24 * 60 * 60 * 1000).toISOString(),
  },
});
} catch (error) {
  console.error('Pricing error:', error);
  return NextResponse.json(
    { error: 'Internal server error' },
    { status: 500 }
  );
}
}

function calculateDeliveryTime(material: string, quantity: number): number {
  const baseDays = {
    canvas: 8,
    metal: 12,
    acrylic: 14,
    paper: 6,
    wood: 11,
  };

  let days = baseDays[material] || 10;

  // Agregar días por volumen
  if (quantity > 5) days += 3;
  if (quantity > 10) days += 5;

  return days;
}

```

4.3 Ejemplo de Cálculo

Caso: Canvas 24"x36" (864 pulgadas²)

1. Área: $24 \times 36 = 864 \text{ pulg}^2$
 2. Precio base Pictorem: $864 \times \$0.12 = \103.68
 3. **Costo** IA: $\$0.02$
 4. Subtotal: $\$103.70$
 5. Markup 2.5x: $\$103.70 \times 2.5 = \259.25
 6. Margen 20%: $\$259.25 \times 0.20 = \51.85
 7. Precio final: $\$259.25 + \$51.85 = \$311.10$
 8. Aplicar límites: $\$311.10$ (dentro de rango $\$25-\500)
- ✓ PRECIO AL CLIENTE: $\$311.10 \text{ USD}$

5. Integración de Pagos

5.1 Proveedores Soportados

Stripe

Configuración:

```
STRIPE_SECRET_KEY=sk_live_...
STRIPE_PUBLISHABLE_KEY=pk_live_...
```

Endpoint: `app/api/payments/stripe/create-session/route.ts`

```

import Stripe from 'stripe';
import { NextResponse } from 'next/server';

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: '2023-10-16',
});

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const { orderId, amount, currency = 'USD' } = body;

    const session = await stripe.checkout.sessions.create({
      payment_method_types: ['card'],
      line_items: [
        {
          price_data: {
            currency: currency.toLowerCase(),
            product_data: {
              name: 'Custom AI Wallpaper',
              description: `Order #${orderId}`,
            },
            unit_amount: Math.round(amount * 100), // Convertir a centavos
          },
          quantity: 1,
        },
      ],
      mode: 'payment',
      success_url: `${process.env.NEXTAUTH_URL}/ai-studio/success?session_id={CHECKOUT_SESSION_ID}`,
      cancel_url: `${process.env.NEXTAUTH_URL}/ai-studio?canceled=true`,
      metadata: {
        orderId,
      },
    });

    return NextResponse.json({ sessionId: session.id, url: session.url });
  } catch (error) {
    console.error('Stripe error:', error);
    return NextResponse.json(
      { error: 'Payment session creation failed' },
      { status: 500 }
    );
  }
}

```

PayPal

Configuración:

```

PAYPAL_CLIENT_ID=...
PAYPAL_CLIENT_SECRET=...
PAYPAL_MODE=live # o 'sandbox' para pruebas

```

Endpoint: `app/api/payments/paypal/create-order/route.ts`

```

import { NextResponse } from 'next/server';

const PAYPAL_API = process.env.PAYPAL_MODE === 'live'
  ? 'https://api-m.paypal.com'
  : 'https://api-m.sandbox.paypal.com';

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const { orderId, amount, currency = 'USD' } = body;

    // Obtener access token
    const auth = Buffer.from(
      `${process.env.PAYPAL_CLIENT_ID}:${process.env.PAYPAL_CLIENT_SECRET}`
    ).toString('base64');

    const tokenResponse = await fetch(`${PAYPAL_API}/v1/oauth2/token`, {
      method: 'POST',
      headers: {
        'Authorization': `Basic ${auth}`,
        'Content-Type': 'application/x-www-form-urlencoded',
      },
      body: 'grant_type=client_credentials',
    });

    const { access_token } = await tokenResponse.json();

    // Crear orden en PayPal
    const orderResponse = await fetch(`${PAYPAL_API}/v2/checkout/orders`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${access_token}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        intent: 'CAPTURE',
        purchase_units: [
          {
            reference_id: orderId,
            amount: {
              currency_code: currency,
              value: amount.toFixed(2),
            },
            description: `Custom AI Wallpaper - Order #${orderId}`,
          },
        ],
        application_context: {
          return_url: `${process.env.NEXTAUTH_URL}/ai-studio/success`,
          cancel_url: `${process.env.NEXTAUTH_URL}/ai-studio?canceled=true`,
        },
      }),
    });

    const orderData = await orderResponse.json();

    return NextResponse.json({
      orderId: orderData.id,
      approvalUrl: orderData.links.find(link => link.rel === 'approve')?.href,
    });
  } catch (error) {
    console.error('PayPal error:', error);
    return NextResponse.json(

```

```

    { error: 'PayPal order creation failed' },
    { status: 500 }
  );
}
}

```

5.2 Flujo de Pago Completo

1. Cliente selecciona método de pago (Stripe o PayPal)

↓

2. Frontend llama a `/api/payments/{provider}/create`

↓

3. Backend crea sesión de pago y retorna URL

↓

4. Cliente es redirigido a página de pago (Stripe/PayPal)

↓

5. Cliente completa pago en plataforma externa

↓

6. Webhook recibe confirmación de pago
POST `/api/payments/{provider}/webhook`

↓

7. Backend actualiza Order status → "paid"
Crea PaymentTransaction con transactionId

↓

8. Trigger automático: Enviar orden a Pictorem
POST `/api/pictorem/sync-order`

↓

9. Cliente redirigido a página de éxito
`/ai-studio/success?session_id=xxx`

6. Proceso de Creación de Órdenes

6.1 Endpoint de Creación

Archivo: `app/api/ai-wallpaper/create-order/route.ts`

```

import { NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';
import { getServerSession } from 'next-auth';
import { authOptions } from '@lib/auth-options';

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const {
      imageUrl,
      width,
      height,
      material,
      quantity,
      customerEmail,
      customerName,
      customerPhone,
      shippingAddress,
      totalAmount,
    } = body;

    // Validación de campos requeridos
    if (!imageUrl || !width || !height || !material || !customerEmail || !shippingAddress) {
      return NextResponse.json(
        { error: 'Missing required fields' },
        { status: 400 }
      );
    }

    // Obtener sesión del usuario (si existe)
    const session = await getServerSession(authOptions);
    const userId = session?.user?.id || null;

    // Generar número de orden único
    const orderNumber = `BWP-${Date.now()}-${Math.random().toString(36).substr(2, 9).toUpperCase()}`;

    // Calcular precio unitario
    const unitPrice = totalAmount / quantity;

    // Crear orden en base de datos
    const order = await prisma.order.create({
      data: {
        orderNumber,
        userId,
        customerEmail,
        customerName,
        customerPhone: customerPhone || null,
        shippingAddress,
        totalAmount,
        status: 'pending', // pending, paid, processing, shipped, delivered, cancelled
        aiWallpaperOrders: {
          create: {
            imageUrl,
            width,
            height,
            material,
            quantity,
            unitPrice,
            totalPrice: totalAmount,
            pictoremStatus: 'pending',
          },
        },
      },
    });
  } catch (error) {
    console.error(error);
    return NextResponse.json(
      { error: 'Internal server error' },
      { status: 500 }
    );
  }
}

```

```

        },
      },
    },
    include: {
      aiWallpaperOrders: true,
    },
  });

  // Retornar orden creada
  return NextResponse.json({
    success: true,
    order: {
      id: order.id,
      orderNumber: order.orderNumber,
      status: order.status,
      totalAmount: order.totalAmount,
      aiWallpaperOrder: order.aiWallpaperOrders[0],
    },
  });
} catch (error) {
  console.error('Order creation error:', error);
  return NextResponse.json(
    { error: 'Failed to create order', details: error.message },
    { status: 500 }
  );
}
}
}

```

6.2 Estructura de Dirección de Envío

```

interface ShippingAddress {
  fullName: string;
  addressLine1: string;
  addressLine2?: string;
  city: string;
  state: string;
  zipCode: string;
  country: string;
  phone?: string;
  instructions?: string; // Instrucciones especiales de entrega
}

```

6.3 Estados de Orden

Estado	Descripción	Acciones Disponibles
pending	Orden creada, esperando pago	Pagar, Cancelar
paid	Pago confirmado	Enviar a Pictorem
processing	Enviada a Pictorem	Ver estado
shipped	En tránsito	Track envío
delivered	Entregada al cliente	Dejar reseña
cancelled	Orden cancelada	Reembolsar (si pagada)
refunded	Dinero devuelto	-

7. Sistema de Envíos y Tracking

7.1 Integración con Pictorem API

Endpoint de Sincronización

Archivo: `app/api/pictorem/sync-order/route.ts`

```

import { NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';

const PICTUREM_API_URL = process.env.PICTUREM_API_URL || 'https://api.pictorem.com/v1';
const PICTUREM_API_KEY = process.env.PICTUREM_API_KEY;

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const { orderId } = body;

    if (!orderId) {
      return NextResponse.json(
        { error: 'Order ID is required' },
        { status: 400 }
      );
    }

    // Obtener orden de la base de datos
    const order = await prisma.order.findUnique({
      where: { id: orderId },
      include: {
        aiWallpaperOrders: true,
      },
    });

    if (!order) {
      return NextResponse.json(
        { error: 'Order not found' },
        { status: 404 }
      );
    }

    if (order.status !== 'paid') {
      return NextResponse.json(
        { error: 'Order must be paid before syncing to Pictorem' },
        { status: 400 }
      );
    }

    const aiOrder = order.aiWallpaperOrders[0];

    if (!aiOrder) {
      return NextResponse.json(
        { error: 'No AI wallpaper order found' },
        { status: 404 }
      );
    }

    // Preparar datos para Pictorem
    const pictoremPayload = {
      preorder_code: order.orderNumber,
      image_url: aiOrder.imageUrl,
      product: {
        type: aiOrder.material,
        width: aiOrder.width,
        height: aiOrder.height,
        unit: 'inch',
      },
      quantity: aiOrder.quantity,
      shipping: {

```

```

    name: order.customerName,
    email: order.customerEmail,
    phone: order.customerPhone,
    address: order.shippingAddress,
  },
  metadata: {
    order_id: order.id,
    order_number: order.orderNumber,
    customer_email: order.customerEmail,
  },
};

// Enviar a Pictorem API
const response = await fetch(`${PICTOREM_API_URL}/orders`, {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${PICTOREM_API_KEY}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify(pictoremPayload),
});

if (!response.ok) {
  const errorData = await response.json();
  throw new Error(`Pictorem API error: ${errorData.message || 'Unknown error'}`);
}

const pictoremResponse = await response.json();

// Actualizar orden con información de Pictorem
await prisma.aiWallpaperOrder.update({
  where: { id: aiOrder.id },
  data: {
    pictoremOrderId: pictoremResponse.order_id,
    pictoremStatus: 'sent',
    pictoremResponse: pictoremResponse,
  },
});

await prisma.order.update({
  where: { id: order.id },
  data: {
    status: 'processing',
  },
});

return NextResponse.json({
  success: true,
  pictoremOrderId: pictoremResponse.order_id,
  status: 'processing',
  estimatedDelivery: pictoremResponse.estimated_delivery_date,
});
} catch (error) {
  console.error('Pictorem sync error:', error);

  // Marcar orden como fallida
  if (body?.orderId) {
    await prisma.aiWallpaperOrder.updateMany({
      where: { orderId: body.orderId },
      data: {
        pictoremStatus: 'failed',
      },
    });
  }
}

```

```
    }  
  
    return NextResponse.json(  
      { error: 'Failed to sync with Pictorem', details: error.message },  
      { status: 500 }  
    );  
  }  
}
```

Endpoint de Tracking

Archivo: app/api/pictorem/tracking/route.ts

```

import { NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';

export async function GET(request: Request) {
  try {
    const { searchParams } = new URL(request.url);
    const orderNumber = searchParams.get('orderNumber');

    if (!orderNumber) {
      return NextResponse.json(
        { error: 'Order number is required' },
        { status: 400 }
      );
    }

    // Buscar orden en base de datos
    const order = await prisma.order.findUnique({
      where: { orderNumber },
      include: {
        aiWallpaperOrders: true,
      },
    });

    if (!order) {
      return NextResponse.json(
        { error: 'Order not found' },
        { status: 404 }
      );
    }

    const aiOrder = order.aiWallpaperOrders[0];

    if (!aiOrder?.pictoremOrderId) {
      return NextResponse.json({
        status: order.status,
        message: 'Order not yet sent to production',
      });
    }

    // Consultar estado en Pictorem
    const response = await fetch(
      `${process.env.PICTUREM_API_URL}/orders/${aiOrder.pictoremOrderId}`,
      {
        headers: {
          'Authorization': `Bearer ${process.env.PICTUREM_API_KEY}`,
        },
      },
    );

    if (!response.ok) {
      throw new Error('Failed to fetch tracking info from Pictorem');
    }

    const trackingData = await response.json();

    // Actualizar tracking number si está disponible
    if (trackingData.tracking_number && !aiOrder.trackingNumber) {
      await prisma.aiWallpaperOrder.update({
        where: { id: aiOrder.id },
        data: {
          trackingNumber: trackingData.tracking_number,
        },
      },
    }

```

```

    });

    if (trackingData.status === 'shipped') {
      await prisma.order.update({
        where: { id: order.id },
        data: { status: 'shipped' },
      });
    }
  }

  return NextResponse.json({
    orderNumber: order.orderNumber,
    status: order.status,
    trackingNumber: trackingData.tracking_number,
    carrier: trackingData.carrier,
    estimatedDelivery: trackingData.estimated_delivery_date,
    trackingUrl: trackingData.tracking_url,
    timeline: trackingData.timeline || [],
  });
} catch (error) {
  console.error('Tracking error:', error);
  return NextResponse.json(
    { error: 'Failed to fetch tracking information' },
    { status: 500 }
  );
}
}
}

```

7.2 Notificaciones de Estado

Email Templates

Orden Confirmada:

```

<h2>¡Orden Confirmada! 🎉</h2>
<p>Hola {{customerName}},</p>
<p>Tu orden <strong>{{orderNumber}}</strong> ha sido confirmada.</p>
<p>Detalles del producto:</p>
<ul>
  <li>Material: {{material}}</li>
  <li>Dimensiones: {{width}}" x {{height}}"</li>
  <li>Cantidad: {{quantity}}</li>
</ul>
<p>Entrega estimada: {{estimatedDelivery}}</p>
<p><a href="{{trackingUrl}}">Rastrear mi orden</a></p>

```

Orden Enviada:

```

<h2>¡Tu orden está en camino! 📦</h2>
<p>Hola {{customerName}},</p>
<p>Tu orden <strong>{{orderNumber}}</strong> ha sido enviada.</p>
<p>Número de rastreo: <strong>{{trackingNumber}}</strong></p>
<p>Transportista: {{carrier}}</p>
<p><a href="{{trackingUrl}}">Rastrear envío</a></p>

```

8. Gestión de Logos y Personalización

8.1 Subida de Logo del Cliente

Endpoint: `app/api/admin/upload/route.ts`

```

import { NextResponse } from 'next/server';
import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';

const s3Client = new S3Client({});
const BUCKET_NAME = process.env.AWS_BUCKET_NAME;

export async function POST(request: Request) {
  try {
    const formData = await request.formData();
    const file = formData.get('file') as File;
    const type = formData.get('type') as string; // 'logo' | 'watermark'

    if (!file) {
      return NextResponse.json(
        { error: 'No file provided' },
        { status: 400 }
      );
    }

    // Validar tipo de archivo
    const allowedTypes = ['image/png', 'image/jpeg', 'image/webp'];
    if (!allowedTypes.includes(file.type)) {
      return NextResponse.json(
        { error: 'Invalid file type. Only PNG, JPEG, and WebP are allowed.' },
        { status: 400 }
      );
    }

    // Validar tamaño (max 5MB)
    if (file.size > 5 * 1024 * 1024) {
      return NextResponse.json(
        { error: 'File too large. Maximum size is 5MB.' },
        { status: 400 }
      );
    }

    // Generar nombre único
    const timestamp = Date.now();
    const filename = `${type}/${timestamp}-${file.name}`;
    const key = `${process.env.AWS_FOLDER_PREFIX}${filename}`;

    // Convertir File a Buffer
    const buffer = Buffer.from(await file.arrayBuffer());

    // Subir a S3
    const command = new PutObjectCommand({
      Bucket: BUCKET_NAME,
      Key: key,
      Body: buffer,
      ContentType: file.type,
      ACL: 'public-read',
    });

    await s3Client.send(command);

    // Generar URL pública
    const url = `https://${BUCKET_NAME}.s3.amazonaws.com/${key}`;

    return NextResponse.json({
      success: true,
      url,
    });
  }
}

```

```
    filename: file.name,  
    size: file.size,  
    type: file.type,  
  });  
} catch (error) {  
  console.error('Upload error:', error);  
  return NextResponse.json(  
    { error: 'Failed to upload file' },  
    { status: 500 }  
  );  
}  
}
```

8.2 Aplicar Logo a Imagen Generada

Implementación con Canvas API (Node.js)

```

import { createCanvas, loadImage } from 'canvas';

async function applyLogoToWallpaper(
  wallpaperUrl: string,
  logoUrl: string,
  position: 'top-left' | 'top-right' | 'bottom-left' | 'bottom-right' | 'center' = 'bottom-right',
  opacity: number = 0.8
): Promise<Buffer> {
  // Cargar imágenes
  const wallpaper = await loadImage(wallpaperUrl);
  const logo = await loadImage(logoUrl);

  // Crear canvas
  const canvas = createCanvas(wallpaper.width, wallpaper.height);
  const ctx = canvas.getContext('2d');

  // Dibujar wallpaper de fondo
  ctx.drawImage(wallpaper, 0, 0);

  // Calcular tamaño del logo (10% del ancho del wallpaper)
  const logoWidth = wallpaper.width * 0.1;
  const logoHeight = (logo.height / logo.width) * logoWidth;

  // Calcular posición
  let x, y;
  const padding = 20;

  switch (position) {
    case 'top-left':
      x = padding;
      y = padding;
      break;
    case 'top-right':
      x = wallpaper.width - logoWidth - padding;
      y = padding;
      break;
    case 'bottom-left':
      x = padding;
      y = wallpaper.height - logoHeight - padding;
      break;
    case 'bottom-right':
      x = wallpaper.width - logoWidth - padding;
      y = wallpaper.height - logoHeight - padding;
      break;
    case 'center':
      x = (wallpaper.width - logoWidth) / 2;
      y = (wallpaper.height - logoHeight) / 2;
      break;
  }

  // Aplicar opacidad
  ctx.globalAlpha = opacity;

  // Dibujar logo
  ctx.drawImage(logo, x, y, logoWidth, logoHeight);

  // Restaurar opacidad
  ctx.globalAlpha = 1.0;

  // Retornar como buffer

```

```
return canvas.toBuffer('image/png');
}
```

8.3 Watermark Automático

Configuración:

```
const WATERMARK_CONFIG = {
  enabled: true,
  text: '© Barrera Wallpaper',
  position: 'bottom-right',
  fontSize: 24,
  color: 'rgba(255, 255, 255, 0.5)',
  font: 'Arial',
};

function applyWatermark(canvas: Canvas, config: typeof WATERMARK_CONFIG) {
  const ctx = canvas.getContext('2d');

  ctx.font = `${config.fontSize}px ${config.font}`;
  ctx.fillStyle = config.color;
  ctx.textAlign = 'right';
  ctx.textBaseline = 'bottom';

  const padding = 20;
  ctx.fillText(
    config.text,
    canvas.width - padding,
    canvas.height - padding
  );
}
```

9. Configuración Técnica

9.1 Variables de Entorno Requeridas

Archivo: `.env`

```

# Base de Datos
DATABASE_URL="postgresql://user:password@host:port/database?pgboun-
cer=true&connection_limit=1"

# NextAuth
NEXTAUTH_SECRET="tu-secret-key-minimo-32-caracteres"
NEXTAUTH_URL="https://barrerawallpaper.com"

# Google AI Studio (Nano Banana)
GOOGLE_AI_STUDIO_API_KEY="AIza..."

# OpenAI (Fallback)
OPENAI_API_KEY="sk-..."

# Pictorem
PICTOREM_API_KEY="tu-api-key"
PICTOREM_API_URL="https://api.pictorem.com/v1"

# Stripe
STRIPE_SECRET_KEY="sk_live..."
STRIPE_PUBLISHABLE_KEY="pk_live..."
STRIPE_WEBHOOK_SECRET="whsec..."

# PayPal
PAYPAL_CLIENT_ID="tu-client-id"
PAYPAL_CLIENT_SECRET="tu-client-secret"
PAYPAL_MODE="live" # o 'sandbox'

# AWS S3
AWS_BUCKET_NAME="tu-bucket-name"
AWS_FOLDER_PREFIX="barrera-wallpaper/"

# Email (SendGrid/SMTP)
EMAIL_FROM="noreply@barrerawallpaper.com"
SMTP_HOST="smtp.sendgrid.net"
SMTP_PORT="587"
SMTP_USER="apikey"
SMTP_PASSWORD="tu-sendgrid-api-key"

```

9.2 Configuración de Prisma

Archivo: prisma/schema.prisma

```

generator client {
  provider = "prisma-client-js"
  binaryTargets = ["native", "linux-musl-openssl-3.0.x"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// Modelos definidos anteriormente...

```

Comandos importantes:

```
# Generar cliente Prisma
yarn prisma generate

# Aplicar cambios al schema
yarn prisma db push

# Seed de datos iniciales
yarn prisma db seed

# Abrir Prisma Studio
yarn prisma studio
```

9.3 Configuración de AWS S3

Política de Bucket (Bucket Policy):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::tu-bucket-name/barrera-wallpaper/public/*"
    },
    {
      "Sid": "AllowAppUpload",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::ACCOUNT_ID:user/barrera-app"
      },
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject"
      ],
      "Resource": "arn:aws:s3:::tu-bucket-name/barrera-wallpaper/*"
    }
  ]
}
```

CORS Configuration:

```
[
  {
    "AllowedHeaders": ["*"],
    "AllowedMethods": ["GET", "PUT", "POST", "DELETE"],
    "AllowedOrigins": ["https://barrerawallpaper.com"],
    "ExposeHeaders": ["ETag"]
  }
]
```

10. Flujos de Datos Detallados

10.1 Flujo Completo: Desde Generación hasta Entrega

FASE 1: GENERACIÓN DE IMAGEN

1. Cliente visita /ai-studio
2. Ingresa prompt, estilo, colores
3. Frontend ☐ POST /api/ai-wallpaper/generate
4. Backend:
 - a. Crea AIGenerationRequest (status: pending)
 - b. Llama a Google AI Studio API
 - c. Si falla ☐ Fallback a OpenAI
 - d. Recibe imagen generada
 - e. Sube imagen a S3
 - f. Actualiza AIGenerationRequest (status: completed, URL de S3)
5. Frontend recibe URL de imagen
6. Cliente ve preview de su diseño

FASE 2: CONFIGURACIÓN Y PRICING

7. Cliente selecciona:
 - Dimensiones (width ☐ height)
 - Material (canvas, metal, acrylic, paper, wood)
 - Cantidad
8. Frontend ☐ POST /api/pictorem/pricing
9. Backend calcula:
 - ☐ Area en pulgadas²
 - Costo base de Pictorem
 - Markup del material
 - Margen de ganancia
 - Descuentos por volumen
 - Tiempo de entrega estimado
10. Frontend muestra:
 - Precio final
 - Breakdown de costos
 - Fecha estimada de entrega

FASE 3: CREACIÓN DE ORDEN

11. Cliente completa formulario:
 - Nombre completo
 - Email
 - Teléfono
 - Dirección de envío
12. Frontend ☐ POST /api/ai-wallpaper/create-order
13. Backend:
 - a. Valida datos
 - b. Genera orderNumber ☐ Único
 - c. Crea Order (status: pending)
 - d. Crea AIWallpaperOrder vinculado
14. Frontend recibe confirmation

FASE 4: PROCESAMIENTO DE PAGO

15. Cliente elige método de pago (Stripe/PayPal)
16. Frontend ☐ POST /api/payments/{provider}/create-session
17. Backend:

- a. Crea sesión de pago
- b. Retorna URL de checkout
- 18. Cliente redirigido a página de pago
- 19. Cliente completa pago
- 20. Webhook → POST /api/payments/{provider}/webhook
- 21. Backend:
 - a. Valida firma del webhook
 - b. Crea PaymentTransaction
 - c. Actualiza Order (status: paid)
 - d. Trigger automático → Enviar a Pictorem

FASE 5: ENVÍO A PICTUREM

- 22. Backend → POST /api/pictorem/sync-order
- 23. Backend prepara payload:
 - preorder_code
 - image_url (S3)
 - product config (material, dimensions)
 - shipping info
- 24. Backend → POST https://api.pictorem.com/v1/orders
- 25. Pictorem API responde:
 - order_id
 - estimated_delivery_date
- 26. Backend actualiza:
 - AIWallpaperOrder.pictoremOrderId
 - AIWallpaperOrder.pictoremStatus = 'sent'
 - Order.status = 'processing'
- 27. Email enviado al cliente: "Orden en producción"

FASE 6: PRODUCCIÓN Y ENVÍO

- 28. Pictorem imprime el wallpaper (5-14 días según material)
- 29. Pictorem empaqueta y envía
- 30. Pictorem actualiza estado:
 - status: 'shipped'
 - tracking_number
 - carrier
- 31. Webhook de Pictorem → POST /api/pictorem/webhook
- 32. Backend actualiza:
 - AIWallpaperOrder.trackingNumber
 - Order.status = 'shipped'
- 33. Email enviado al cliente: "Orden enviada" + tracking

FASE 7: ENTREGA Y SEGUIMIENTO

- 34. Cliente puede rastrear:
 - GET /api/pictorem/tracking?orderNumber=xxx
- 35. Backend consulta:
 - Pictorem API para estado actualizado
 - Carrier API (UPS/FedEx) para tracking detallado
- 36. Frontend muestra:
 - Timeline de envío
 - Ubicación actual
 - Fecha de entrega estimada
- 37. Cuando entrega confirmada:
 - Order.status = 'delivered'
 - Email: "Orden entregada" + solicitud de reseña

FASE 8: POST-ENTREGA

38. Cliente puede:
 - Dejar reseña del producto
 - Subir foto de instalación
 - Solicitar soporte si hay problemas
39. Si hay problema:
 - Cliente contacta soporte
 - Admin revisa orden
 - Se coordina reimpresión o reembolso

11. Manejo de Errores

11.1 Estrategias de Error Handling

Generación de IA

```
// Retry con exponential backoff
async function generateWithRetry(prompt: string, maxRetries = 3): Promise<string> {
  let lastError: Error;

  for (let i = 0; i < maxRetries; i++) {
    try {
      const imageUrl = await generateImage(prompt);
      return imageUrl;
    } catch (error) {
      lastError = error;
      const delay = Math.pow(2, i) * 1000; // 1s, 2s, 4s
      await new Promise(resolve => setTimeout(resolve, delay));
    }
  }

  throw new Error(`Failed after ${maxRetries} retries: ${lastError.message}`);
}
```

Sincronización con Pictorem

```
// Endpoint para reintentar órdenes fallidas
export async function POST(request: Request) {
  const { orderId } = await request.json();

  try {
    // Intentar sync nuevamente
    await syncOrderToPictorem(orderId);

    return NextResponse.json({ success: true });
  } catch (error) {
    // Registrar en sistema de logs
    await logError({
      type: 'pictorem_sync_failed',
      orderId,
      error: error.message,
      timestamp: new Date(),
    });

    // Notificar al admin
    await sendAdminAlert({
      subject: `Pictorem Sync Failed - Order ${orderId}`,
      body: error.message,
    });

    return NextResponse.json(
      { error: 'Sync failed, admin notified' },
      { status: 500 }
    );
  }
}
```

Pagos

```
// Manejo de webhooks duplicados
const processedWebhooks = new Set<string>();

export async function POST(request: Request) {
  const signature = request.headers.get('stripe-signature');
  const body = await request.text();

  try {
    const event = stripe.webhooks.constructEvent(body, signature, WEBHOOK_SECRET);

    // Evitar procesamiento duplicado
    if (processedWebhooks.has(event.id)) {
      console.log('Webhook already processed:', event.id);
      return NextResponse.json({ received: true });
    }

    processedWebhooks.add(event.id);

    // Procesar evento
    await handleWebhookEvent(event);

    return NextResponse.json({ received: true });
  } catch (error) {
    console.error('Webhook error:', error);
    return NextResponse.json(
      { error: 'Webhook handling failed' },
      { status: 400 }
    );
  }
}
```

11.2 Códigos de Error Estandarizados

```
enum ErrorCode {
  // Generación de IA
  AI_GENERATION_FAILED = 'AI_GENERATION_FAILED',
  AI_PROVIDER_UNAVAILABLE = 'AI_PROVIDER_UNAVAILABLE',
  AI_INVALID_PROMPT = 'AI_INVALID_PROMPT',

  // Pictorem
  PICTUREM_API_ERROR = 'PICTOREM_API_ERROR',
  PICTUREM_INVALID_CONFIG = 'PICTOREM_INVALID_CONFIG',
  PICTUREM_SYNC_FAILED = 'PICTOREM_SYNC_FAILED',

  // Pagos
  PAYMENT_FAILED = 'PAYMENT_FAILED',
  PAYMENT_WEBHOOK_INVALID = 'PAYMENT_WEBHOOK_INVALID',
  PAYMENT_DUPLICATE = 'PAYMENT_DUPLICATE',

  // Órdenes
  ORDER_NOT_FOUND = 'ORDER_NOT_FOUND',
  ORDER_INVALID_STATUS = 'ORDER_INVALID_STATUS',
  ORDER_CREATION_FAILED = 'ORDER_CREATION_FAILED',

  // Validación
  VALIDATION_ERROR = 'VALIDATION_ERROR',
  MISSING_REQUIRED_FIELD = 'MISSING_REQUIRED_FIELD',
  INVALID_DIMENSIONS = 'INVALID_DIMENSIONS',
}

interface ApiError {
  code: ErrorCode;
  message: string;
  details?: any;
  timestamp: string;
}
```

12. Seguridad y Validación

12.1 Validación de Inputs

```
import { z } from 'zod';

// Schema de validación para creación de orden
const createOrderSchema = z.object({
  imageUrl: z.string().url(),
  width: z.number().min(8).max(120),
  height: z.number().min(8).max(120),
  material: z.enum(['canvas', 'metal', 'acrylic', 'paper', 'wood']),
  quantity: z.number().int().min(1).max(100),
  customerEmail: z.string().email(),
  customerName: z.string().min(2).max(100),
  customerPhone: z.string().optional(),
  shippingAddress: z.object({
    addressLine1: z.string().min(5),
    city: z.string().min(2),
    state: z.string().min(2),
    zipCode: z.string().regex(/^\d{5}(-\d{4})?$/),
    country: z.string().length(2),
  }),
});

// Uso en endpoint
export async function POST(request: Request) {
  try {
    const body = await request.json();

    // Validar con Zod
    const validatedData = createOrderSchema.parse(body);

    // Continuar con lógica...
  } catch (error) {
    if (error instanceof z.ZodError) {
      return NextResponse.json(
        { error: 'Validation failed', details: error.errors },
        { status: 400 }
      );
    }
    throw error;
  }
}
```

12.2 Rate Limiting

```
import { RateLimiter } from 'limiter';

// Limitar generación de IA: 5 requests por minuto por IP
const aiLimiter = new RateLimiter({
  tokensPerInterval: 5,
  interval: 'minute',
});

export async function POST(request: Request) {
  const ip = request.headers.get('x-forwarded-for') || 'unknown';

  const hasTokens = await aiLimiter.removeTokens(1);

  if (!hasTokens) {
    return NextResponse.json(
      { error: 'Rate limit exceeded. Please try again later.' },
      { status: 429 }
    );
  }

  // Continuar con generación...
}
```

12.3 Sanitización de Datos

```
import DOMPurify from 'isomorphic-dompurify';

function sanitizeUserInput(input: string): string {
  // Remover HTML/JavaScript
  const clean = DOMPurify.sanitize(input, {
    ALLOWED_TAGS: [],
    ALLOWED_ATTR: [],
  });

  // Limitar longitud
  return clean.substring(0, 1000);
}

// Uso
const safePrompt = sanitizeUserInput(userPrompt);
```

13. Monitoring y Analytics

13.1 Métricas Clave

```
interface Metrics {  
  // Generación de IA  
  totalGenerations: number;  
  successfulGenerations: number;  
  failedGenerations: number;  
  averageGenerationTime: number; // segundos  
  
  // Órdenes  
  totalOrders: number;  
  paidOrders: number;  
  processingOrders: number;  
  shippedOrders: number;  
  deliveredOrders: number;  
  cancelledOrders: number;  
  
  // Revenue  
  totalRevenue: number;  
  averageOrderValue: number;  
  
  // Pictorem  
  pictorem_synced: number;  
  pictorem_failed: number;  
  pictorem_pending: number;  
  
  // Performance  
  apiResponseTime: number; // ms  
  errorRate: number; // porcentaje  
}
```

13.2 Dashboard de Admin

Archivo: app/admin/dashboard/page.tsx

```

export default async function AdminDashboard() {
  const metrics = await prisma.$queryRaw`
    SELECT
      COUNT(*) FILTER (WHERE status = 'delivered') as delivered_orders,
      COUNT(*) FILTER (WHERE status = 'processing') as processing_orders,
      SUM("totalAmount") FILTER (WHERE status IN ('paid', 'processing', 'shipped',
'delivered')) as total_revenue,
      AVG("totalAmount") as avg_order_value
    FROM "Order"
    WHERE "createdAt" >= NOW() - INTERVAL '30 days'
  `;

  return (
    <div>
      <h1>Admin Dashboard</h1>
      <div className="grid grid-cols-4 gap-4">
        <MetricCard
          title="Total Revenue (30d)"
          value={`$${metrics.total_revenue}`}
          icon={DollarSign}
        />
        <MetricCard
          title="Delivered Orders"
          value={metrics.delivered_orders}
          icon={Package}
        />
        {/* Más métricas... */}
      </div>
    </div>
  );
}

```

14. Troubleshooting

14.1 Problemas Comunes y Soluciones

Problema: “Generación de IA falla constantemente”

Diagnóstico:

```

# Verificar API keys
echo $GOOGLE_AI_STUDIO_API_KEY
echo $OPENAI_API_KEY

# Probar manualmente
curl -X POST https://i.ytimg.com/vi/TmpYMPZvj3Q/hq720.jpg?sqp=-oaymwEhCK4-
FEIIDSFryq4qpAxMIARUAAAAAGAEIAADIQj0AgKJD&rs=A0n4CLDfWyMnX3JXZuhFwpynCJu6J_g1_Q \
  -H 'Content-Type: application/json' \
  -d '{"prompt": "test"}'

```

Soluciones:

1. Verificar que las API keys sean válidas
2. Revisar límites de rate en las plataformas

3. Confirmar que las cuentas tengan créditos
4. Revisar logs de error para detalles específicos

Problema: “Órdenes no se sincronizan con Pictorem”

Diagnóstico:

```
// Revisar órdenes pendientes
const pendingOrders = await prisma.aIWallpaperOrder.findMany({
  where: {
    pictoremStatus: 'pending',
    order: { status: 'paid' },
  },
  include: { order: true },
});

console.log('Pending sync:', pendingOrders.length);
```

Soluciones:

1. Verificar `PICTOREM_API_KEY` en `.env`
2. Probar conexión a Pictorem API manualmente
3. Revisar formato del payload
4. Usar endpoint `/api/pictorem/retry-order` para reintentar

Problema: “Pagos completados pero orden sigue en ‘pending’”

Diagnóstico:

```
// Buscar transacciones sin orden asociada
const orphanedTransactions = await prisma.paymentTransaction.findMany({
  where: {
    status: 'completed',
    order: {
      status: 'pending',
    },
  },
});
```

Soluciones:

1. Verificar que webhooks estén configurados correctamente
2. Revisar logs de webhooks en Stripe/PayPal
3. Manualmente actualizar órdenes con script:

```
// fix-payment-orders.ts
for (const tx of orphanedTransactions) {
  await prisma.order.update({
    where: { id: tx.orderId },
    data: { status: 'paid' },
  });

  // Trigger sync a Pictorem
  await syncOrderToPictorem(tx.orderId);
}
```

15. Conclusiones y Recomendaciones

15.1 Estado Actual de la Implementación

✅ **Completado:**

- Generación de imágenes con IA (Google AI Studio + OpenAI fallback)
- Cálculo dinámico de precios con múltiples materiales
- Sistema de órdenes completo
- Integración de pagos (Stripe + PayPal)
- Base de datos estructurada con Prisma
- Almacenamiento de imágenes en AWS S3
- Panel de administración básico

⚠️ **Parcialmente Implementado:**

- Sincronización automática con Pictorem API (requiere credenciales reales)
- Sistema de tracking de envíos (depende de webhooks de Pictorem)
- Notificaciones por email (requiere configuración SMTP)

🔄 **Pendiente:**

- Aplicación de logos personalizados a las imágenes
- Sistema de reseñas de clientes
- Analytics avanzado
- Programa de descuentos y cupones

15.2 Próximos Pasos Recomendados

Corto Plazo (1-2 semanas)

1. **Configurar credenciales de producción:**

- Obtener API key real de Pictorem
- Configurar Stripe/PayPal en modo Live
- Configurar servicio de email (SendGrid)

2. **Testing exhaustivo:**

- Probar flujo completo end-to-end
- Validar cálculos de precios
- Verificar sincronización con Pictorem

3. **Configurar monitoring:**

- Sentry para tracking de errores
- Google Analytics para métricas
- Alertas por email para errores críticos

Mediano Plazo (1-2 meses)

1. **Optimizaciones:**

- Cachear precios de Pictorem
- Implementar CDN para imágenes
- Optimizar queries de base de datos

2. **Nuevas funcionalidades:**

- Galería de diseños populares

- Sistema de favoritos
- Compartir diseños en redes sociales

3. **Marketing:**

- SEO optimization
- Email marketing automation
- Programa de referidos

Largo Plazo (3-6 meses)

1. **Escalabilidad:**

- Migrar a arquitectura serverless
- Implementar queue system (Bull/BullMQ)
- Caching con Redis

2. **Expansión:**

- Nuevos materiales de impresión
- Integración con más proveedores
- Marketplace de diseñadores

15.3 Costos Estimados de Operación

Por orden:

- Generación de IA: \$0.02
- Almacenamiento S3: \$0.001
- Procesamiento de pago: \$0.30 + 2.9%
- Hosting (Hostinger): ~\$0.10
- Email: \$0.01

Total overhead por orden: ~\$0.43 + 2.9%

Ejemplo con orden de \$311:

- Overhead: $\$0.43 + \$9.02 = \$9.45$
 - Costo Pictorem: ~\$104
 - Margen bruto: $\$311 - \$104 - \$9.45 = \mathbf{\$197.55 (63\%)}$
-

16. Glosario Técnico

Término	Definición
Pictorem	Proveedor de impresión on-demand para productos personalizados
Dropshipping	Modelo de negocio donde no se mantiene inventario físico
AI Generation	Proceso de crear imágenes usando modelos de inteligencia artificial
Nano Banana	Nombre del modelo Gemini 2.5 Flash Image de Google AI Studio
Webhook	Endpoint HTTP que recibe notificaciones automáticas de eventos
Prisma ORM	Object-Relational Mapping para TypeScript/JavaScript
S3	Simple Storage Service de AWS para almacenar archivos
Rate Limiting	Técnica para limitar número de requests por tiempo
Fallback	Sistema alternativo que se activa cuando el principal falla
Markup	Multiplicador aplicado al costo base para calcular precio de venta

17. Referencias y Documentación

APIs Externas

- **Google AI Studio:** <https://ai.google.dev/docs>
- **OpenAI API:** <https://platform.openai.com/docs>
- **Pictorem API:** <https://api.pictorem.com/docs> (requiere credenciales)
- **Stripe API:** <https://stripe.com/docs/api>
- **PayPal API:** <https://developer.paypal.com/docs/api>
- **AWS S3 SDK:** <https://docs.aws.amazon.com/AWSJavaScriptSDK/v3/latest/clients/client-s3>

Frameworks y Librerías

- **Next.js 14:** <https://nextjs.org/docs>
 - **Prisma:** <https://www.prisma.io/docs>
 - **NextAuth.js:** <https://next-auth.js.org/getting-started/introduction>
 - **Tailwind CSS:** <https://tailwindcss.com/docs>
 - **Zod:** <https://zod.dev/>
-

Documento preparado por: Sistema DeepAgent

Fecha: Enero 2026

Versión: 1.0

Cliente: Barrera Wallpaper

Proyecto: barrerawallpaper.com

Este documento es confidencial y propiedad de Barrera Wallpaper. Todos los derechos reservados.